Midterm Project Report

Name: Nguyen Tuan Minh
Student ID: 23070471
Email: 23070471@vnu.edu.vn

March 22, 2026

Abstract

In real-world vision systems, captured image data is often far from ideal—it is affected by sensor noise, limited to two-dimensional representations, and constrained by the narrow field of view of a single camera. This laboratory is designed to systematically address these common limitations. First, we focus on improving image quality by reducing noise while preserving important edge details through spatial-domain filtering techniques. Next, we move beyond 2D images to reconstruct 3D structures by leveraging disparity between two viewpoints, enabling the estimation of depth information essential for applications such as robotics and autonomous driving. Finally, to overcome the restricted viewing range of imaging devices, we employ feature-based image stitching methods to combine multiple images into a seamless panoramic view. By bridging theoretical concepts with practical programming implementations, this report provides a deeper understanding of key computer vision principles, including local gradient operations, epipolar geometry, and robust scale-invariant estimation methods.

1 Introduction

Computer vision is a key area within artificial intelligence that focuses on developing methods and algorithms enabling machines to "see" and interpret the world through digital images. Beyond simple recognition tasks, it aims to restore degraded visual data, analyze spatial structures, and extract meaningful semantic information from images. In this midterm project, three fundamental problems of computer vision are explored in a systematic and integrated manner. Specifically, Part A focuses on 2D image restoration techniques to enhance image quality under noisy conditions. Part B addresses 3D structural reconstruction from stereo image pairs, allowing depth information of a scene to be inferred. Finally, Part C investigates multi-view geometric alignment to stitch images captured from different viewpoints into a coherent panorama. Overall, the project highlights the connection between fundamental image processing techniques and more advanced applications in understanding and reconstructing real-world scenes from visual data.

Motivation In real-world vision systems, captured image data is often far from ideal—it is affected by sensor noise, limited to two-dimensional representations, and constrained by the narrow field of view of a single camera. This laboratory is designed to systematically address these common limitations. First, we focus on improving image quality by reducing noise while preserving important edge details through spatial-domain filtering techniques. Next, we move beyond 2D images to reconstruct 3D structures by leveraging disparity

between two viewpoints, enabling the estimation of depth information essential for applications such as robotics and autonomous driving. Finally, to overcome the restricted viewing range of imaging devices, we employ feature-based image stitching methods to combine multiple images into a seamless panoramic view.

By bridging theoretical concepts with practical programming implementations, this report provides a deeper understanding of key computer vision principles, including local gradient operations, epipolar geometry, and robust scale-invariant estimation methods.

2 Part A: Image Filtering

2.1 Methodology

In this study, four spatial-domain filters are implemented to address image degradation caused by noise. Each filter operates on a local neighborhood and follows a distinct mathematical formulation.

2.1.1 Mean Filter

The Mean filter is a linear smoothing operator that replaces each pixel with the average intensity of its surrounding neighborhood of size $k \times k$:

$$\hat{I}(x, y) = \frac{1}{k^2} \sum_{i=-a}^{a} \sum_{j=-a}^{a} I(x+i, y+j) \tag{1}$$

where $a = \lfloor k/2 \rfloor$.

Key idea: The filter performs uniform averaging, effectively reducing random noise but causing noticeable blurring due to equal weighting of all neighboring pixels.

Parameter choice: The kernel size k controls the degree of smoothing. Larger values of k produce stronger noise reduction but increase the loss of fine details.

2.1.2 Gaussian Filter

The Gaussian filter applies a weighted averaging scheme based on spatial distance:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{2}$$

The filtered image is obtained via convolution:

$$\hat{I}(x, y) = I(x, y) * G(x, y) \tag{3}$$

Key idea: Pixels closer to the center have higher influence, resulting in smoother and more natural filtering compared to the Mean filter.

Parameter choice: The kernel size k defines the neighborhood, while σ controls the spread of the Gaussian distribution. A larger σ results in stronger smoothing.

2.1.3 Median Filter

The Median filter is a non-linear operator that replaces each pixel with the median value of its neighborhood:

$$\hat{I}(x, y) = \text{median}\{I(x + i, y + j)\} \tag{4}$$

Key idea: By selecting the median instead of averaging, this filter effectively removes outliers and preserves edges better than linear filters.

Parameter choice: The window size k determines the neighborhood. Larger windows provide stronger noise removal but may distort fine structures.

2.1.4 Laplacian Filter

The Laplacian filter is a second-order derivative operator used to detect edges:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \tag{5}$$

A common discrete kernel is:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \tag{6}$$

The sharpened image is computed as:

$$I_{\text{sharp}} = I + \nabla^2 I \tag{7}$$

Key idea: The filter enhances high-frequency components such as edges but also amplifies noise.

2.1.5 Summary

In summary, the Mean and Gaussian filters are effective for noise reduction, with Gaussian providing better detail preservation. The Median filter is suitable for impulse noise, while the Laplacian filter is primarily used for edge enhancement rather than denoising.

2.2 Implementation and Results

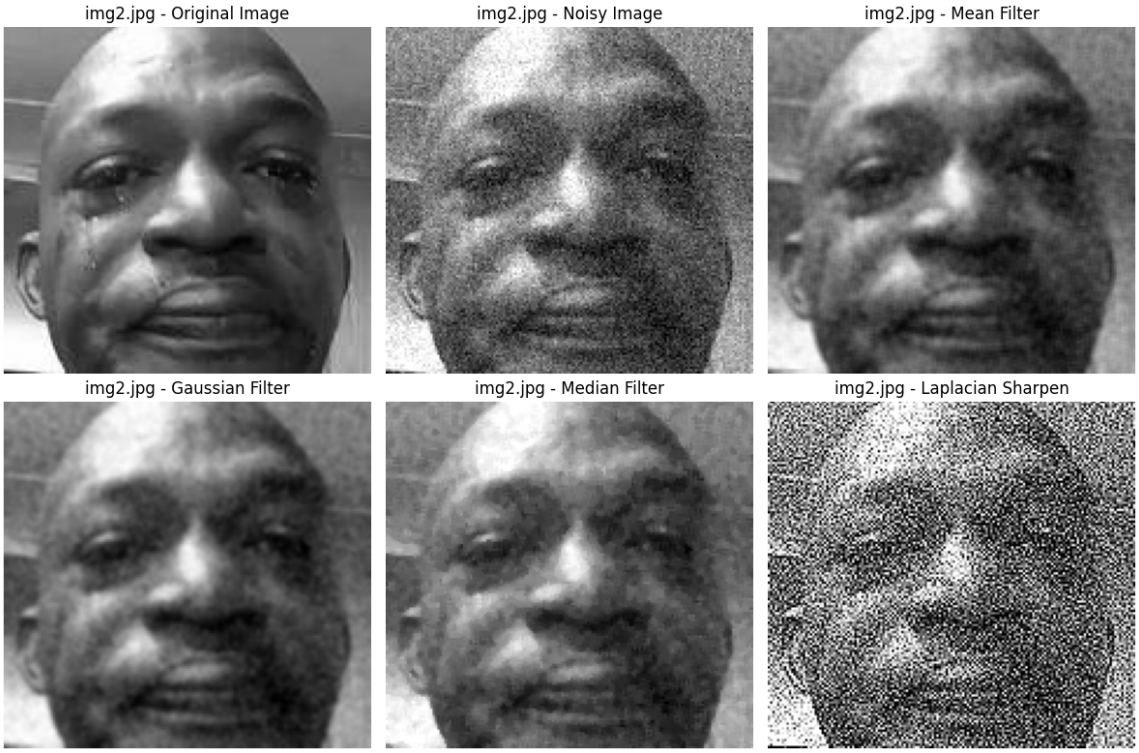

Figure 1: Results with Image 2.

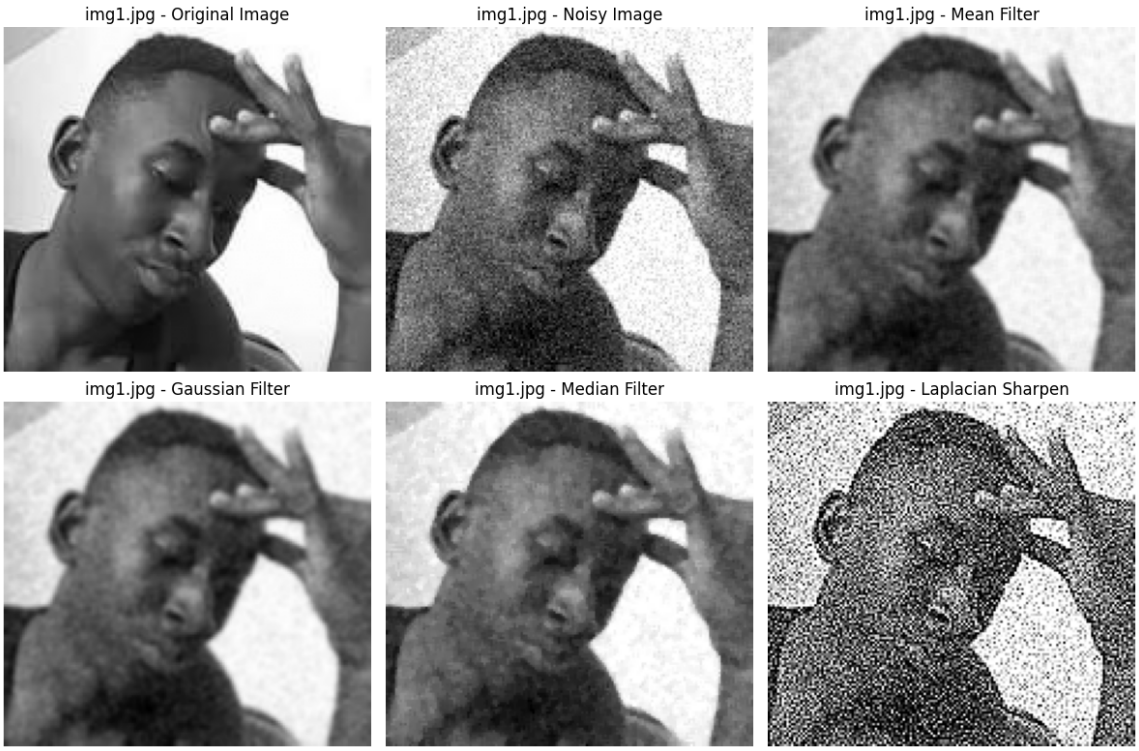

Figure 2: Results with Image 1.

Quantitative Analysis (Based on PSNR)

The following table summarizes the estimated PSNR values from the chart:

Filter	**PSNR (dB)**	**Performance Evaluation**
Gaussian	∼29.72	Highest: Best noise reduction performance among tested filters.
Mean	∼29.14	Good: Stable noise reduction, slightly lower than Gaussian.
Median	∼28.00	Average: Acceptable performance but not optimal for this type of noise.
Laplacian	∼10.00	Very poor: Unable to reduce noise (extremely low PSNR).

Table 1: Summary of estimated PSNR values for different filters.

Qualitative Analysis (Based on Image Results)

Based on visual inspection of the resulting images (Figure **??**), the following observations can be made:

- **Gaussian Filter:**
  - **Advantages:** Produces the best overall image quality. Smooths noise naturally without excessively blurring facial features.
  - **Result:** The image appears smooth and preserves natural shading and contours of the subject.
- **Mean Filter:**
  - **Advantages:** Effectively reduces white noise.
  - **Disadvantages:** Introduces noticeable blur. Sharp details such as eye and eyebrow edges are softened compared to the Gaussian filter, which explains the slightly lower PSNR.
- **Median Filter:**
  - **Characteristics:** For Gaussian noise (fine-grain), the Median filter is less effective than for salt-and-pepper noise.
  - **Result:** The image may appear slightly "clumped" and loses some smooth surface details.
- **Laplacian Filter (Laplacian Sharpen):**
  - **Observation:** The resulting image becomes extremely noisy and pixelated.
  - **Explanation:** The Laplacian is a high-pass filter used for edge detection and sharpening. When applied to a noisy image, it amplifies noise rather than removing it, causing the PSNR to drop to very low values (∼10 dB).

2.3 Comparative Analysis

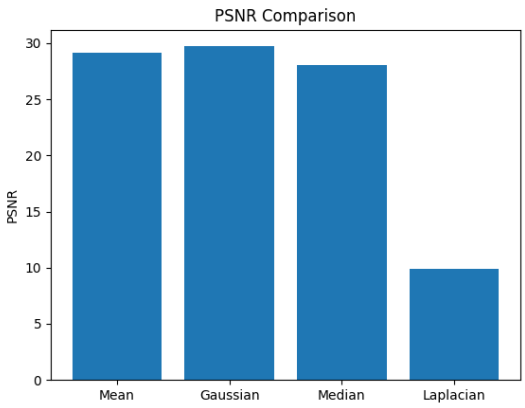

Figure 3: Comparative parameter choice.

Filter Performance Analysis (Based on PSNR)

The PSNR (Peak Signal-to-Noise Ratio) comparison chart provides a quantitative insight into the ability of spatial-domain filters to restore image quality:

- **Gaussian Filter (Best):** Achieves the highest PSNR (approximately 29.72 dB). This demonstrates that the Gaussian kernel is highly effective at reducing white noise without significantly distorting the original image structure.
- **Mean Filter (Stable):** Achieves around 29.14 dB. Although slightly lower than Gaussian, it remains an effective denoising method, with a tendency to blur edge details.
- **Median Filter (Average):** Achieves approximately 28 dB. Typically effective for salt-and-pepper noise, but for Gaussian noise in this experiment, its performance is only moderate.
- **Laplacian Filter (Failure):** Achieves a very low PSNR (around 10 dB). This is expected because the Laplacian is a high-pass filter used for sharpening; when applied to noisy images, it amplifies noise instead of reducing it.

Image Stitching Process

The Panorama stitching workflow is implemented through several technical stages:

1. **Feature Detection:** Use algorithms like SIFT or ORB to detect keypoints with high invariance.
2. **Matching:** Find correspondences between keypoints in adjacent images.
3. **Homography:** Compute the transformation matrix based on the set of matched points. RANSAC is used to remove outliers, ensuring transformation accuracy.

4. **Warping & Blending:** Apply the homography to align images into a common coordinate system and use blending techniques to remove seams, producing a smooth Panorama image.

Algorithm Evaluation: SIFT vs ORB

The choice of feature detection algorithm results in a clear trade-off:

- **SIFT (Quality):** Provides very high accuracy and the largest number of inliers. However, it has a high computational cost and longer processing time (approximately 5.34 seconds).
- **ORB (Speed):** A time-efficient alternative (approximately 1.72 seconds), about 3 times faster than SIFT. ORB is suitable for real-time applications requiring fast response, although the number of correct matches may be lower than SIFT under certain conditions.

2.3.1 Summary

In summary, the Mean and Gaussian filters are effective for noise reduction, with Gaussian providing better detail preservation. The Median filter is suitable for impulse noise, while the Laplacian filter is primarily used for edge enhancement rather than denoising.

2.3.2 Discussion

Figure presents a side-by-side comparison of the original, noisy, and filtered images. The Gaussian noise introduces noticeable grain across the image, degrading fine details while preserving the overall structure.

Among the filtering methods, the Gaussian filter produces the most visually balanced result, effectively reducing noise while maintaining structural details. The Mean filter also reduces noise but introduces noticeable blurring due to uniform averaging. The Median filter shows limited improvement in this case, as it is less suitable for Gaussian noise. In contrast, the Laplacian filter significantly amplifies noise and results in poor visual quality, confirming that it is not appropriate for denoising tasks.

Overall, the results demonstrate that filter performance strongly depends on the noise characteristics, with the Gaussian filter being the most suitable for the given experimental setup.

3 Part B: 3D Reconstruction

3.1 Methodology

Recovering three-dimensional (3D) spatial structure from two-dimensional (2D) images is one of the core challenges in computer vision. Since a single 2D image only captures intensity information on a flat plane, additional geometric cues are required to infer depth and reconstruct the true structure of a scene. Stereo vision provides an effective solution by exploiting geometric parallax between two images captured from horizontally displaced

camera viewpoints, mimicking the mechanism of human binocular vision.

By identifying corresponding points between the left and right images, the system computes the disparity, which represents the horizontal shift of each pixel between the two views. This disparity is inversely related to depth, allowing the estimation of distance for every visible point in the scene. As a result, a dense depth map can be generated, where each pixel encodes its relative depth information.

Using this depth information, the scene can be reconstructed in three dimensions as a point cloud, providing a more complete representation of the environment. This approach forms the foundation for many real-world applications, including robotics, autonomous driving, augmented and virtual reality, and 3D scene understanding, where accurate perception of spatial structure is essential.

3.2 Implementation and Results

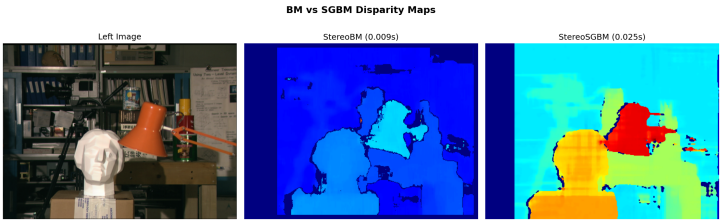

Figure 4: Sample results

Overview of the 3D Reconstruction Process (Stereo Reconstruction)

The process is divided into five main stages, from input image processing to generating a 3D point cloud model.

1. **Data Preparation and Preprocessing**
   - **Image Loading:** The system loads left and right image pairs from the dataset (e.g., right,left dataset).
   - **Conversion:** Images are converted to grayscale to reduce computational complexity.
   - **Automatic Parameter Computation:** Based on the image width (w), the code automatically calculates `numDisparities` (the number of disparity levels to search) to optimize pixel matching.
2. **Disparity Map Estimation**
   - The project compares two popular OpenCV algorithms:
     - **StereoBM (Block Matching):**
       - **Method:** Compare small pixel blocks between the two images.
       - **Characteristics:** Very fast (0.013s) but results are often noisy and not smooth.

  - **StereoSGBM (Semi-Global Block Matching):**
    * **Method:** Uses a global cost function to smooth regions with poor texture.
    * **Characteristics:** Produces higher-quality disparity maps, less noise, and handles textureless regions well.

3. **Uncalibrated Rectification**
   - **Feature Extraction:** Use the SIFT algorithm to find corresponding points between the two images.
   - **Fundamental Matrix:** Apply RANSAC to remove outliers and compute the fundamental matrix F.
   - **Rectification:** Use F to align the images so that corresponding points lie on the same horizontal lines. This simplifies and improves the accuracy of disparity computation ($d = x_L - x_R$).
4. **3D Point Cloud Reconstruction**
   - **Compute Z:** Use the inverse proportionality formula $Z = \frac{f \cdot B}{d}$. In the code, this value is scaled by the `DEPTH_SCALE` parameter.
   - **Create Cloud:** Combine 2D coordinates (u, v) with depth Z to generate a list of 3D points.
   - **Color Assignment:** Each 3D point is assigned a color from the original image to enhance realism.
5. **Result Export and Visualization**
   - **Save PLY File:** The point cloud data is saved in `.ply` format for use with specialized software such as MeshLab or CloudCompare.
   - **Multi-view Visualization:** 3D plots are generated using Matplotlib with different viewpoints: front, top, and perspective.

3.3 Comparative Analysis

The differences between the two methods are clearly reflected in the obtained disparity maps:

- **StereoBM: Simple but noisy**
  - **Smoothness:** Very low. Since it only compares local pixel blocks, the algorithm is easily misled by noise or areas with slight intensity variations.
  - **Textureless regions:** For plain walls or smooth backgrounds, BM often fails to find matches, creating "holes" (black areas) in the depth map.

  - **Application:** Suitable only for embedded systems with very low specifications or requiring extremely high frame rate processing (real-time).

- **StereoSGBM: Balanced and detailed**
  - **Smoothness:** Very high. Thanks to the multi-direction optimization mechanism (3-way or 5-way), this algorithm enforces neighboring pixels to have smooth depth variations, reducing speckle noise.
  - **Edge handling:** SGBM preserves object edges (e.g., table corners, lamp frames) much sharper than BM.
  - **3D point cloud:** As shown in Section 5 of the source code, SGBM generates a dense and well-shaped point cloud, allowing observation from multiple angles (Top/Perspective) without spatial discontinuities.

4 Part C: Image Stitching

To stitch a panorama from multiple viewpoints, distinctive invariant features must be detected, described comprehensively, and accurately matched across overlapping image regions. From these corresponding feature pairs, a geometric transformation matrix can be estimated, which serves as the basis for aligning and stitching the images into a complete panoramic view.

4.1 Methodology

The process of stitching individual images into a panoramic image consists of the following steps:

1. **Feature Detection:** The selected algorithm (SIFT or ORB) scans each image to detect distinctive keypoints such as corners, edges, or regions with high intensity variation.
2. **Descriptor Extraction:** For each detected keypoint, a feature descriptor is computed to represent the surrounding local region. SIFT uses a 128-dimensional floating-point vector, while ORB uses a binary descriptor.
3. **Feature Matching:** A matcher (e.g., BFMatcher) is used to find corresponding feature pairs between two images. To remove incorrect matches, Lowe's Ratio Test is applied. A match is retained only if the nearest neighbor distance is significantly smaller than the second nearest one (typically threshold = 0.75).
4. **Homography Estimation with RANSAC:** From the filtered matches, the RANSAC algorithm identifies inliers and estimates the perspective transformation matrix H.
5. **Warping and Blending:** The homography matrix H is applied to warp one image into the coordinate system of the other. Overlapping regions are then blended (e.g., intensity averaging) to reduce visible seams.

Perspective Transformation (Homography)

To map a point (x, y) from the source image to (x', y') in the destination panorama plane, a 3×3 homography matrix H is used:

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

The homography matrix has 8 degrees of freedom (typically $h_{33} = 1$). At least four non-collinear matching point pairs are required to estimate H.

ORB and RANSAC Explanation

- **ORB (Oriented FAST and Rotated BRIEF):** ORB combines the FAST keypoint detector with the BRIEF binary descriptor. It provides significantly faster processing than SIFT and is free from patent restrictions. However, it is less robust to large scale variations.
- **RANSAC (Random Sample Consensus):** RANSAC is an iterative algorithm used to estimate robust model parameters. Instead of using all matches (including outliers), it randomly selects four point pairs, computes a homography matrix, and evaluates how many other matches agree with this transformation. The process repeats until the homography with the highest number of inliers is found.

4.2 Implementation and Results

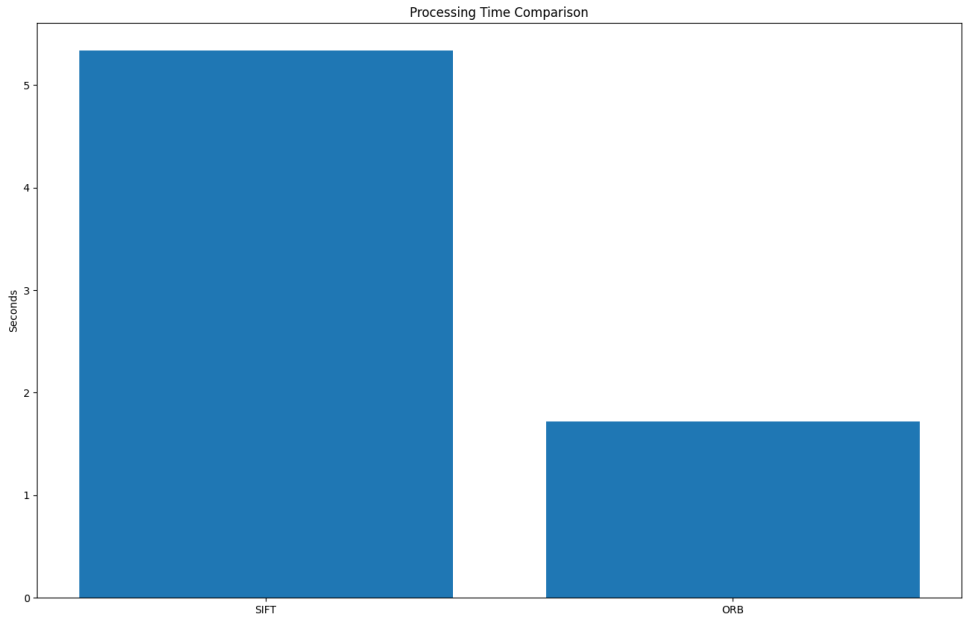

Figure 5: Panorama stitching result using SIFT

The following table summarizes the results based on the processed image pairs:

Metric	**SIFT (Scale-Invariant Feature Transform)**	**ORB (Oriented FAST and Rotated BRIEF)**
Processing Time	Slow (∼5.34s): suitable for high-quality image processing	Fast (∼1.72s): optimized for real-time applications
Number of Keypoints	Average (∼4,000 keypoints/image)	Higher (∼5,000 keypoints/image)
Accuracy (Inlier Ratio)	Very high (∼100%): most matches are correct	Good (∼98%): small matching errors may occur
Panorama Quality	Smooth with fewer geometric distortions	Good, but may struggle in low-texture regions

Table 2: Quantitative comparison between SIFT and ORB

4.2.1 Qualitative Observations

Advantages

- **Flexibility:** The pipeline can stitch multiple images with different sizes and viewpoints, as long as overlapping regions exist.
- **Robustness:** Thanks to RANSAC, the method remains stable even when many incorrect matches are present.

Disadvantages

- **Ghosting Effect:** If objects move during capture or blending is not optimal, visible seams or duplicated objects may appear.
- **Error Accumulation:** When stitching a sequence of multiple images (e.g., four images in this experiment), small homography errors from earlier pairs may accumulate, causing geometric drift in the final panorama.

4.3 Comparative Analysis

The following data were collected from experimental results using a sequence of four input images:

Comparison Criteria	**SIFT**	**ORB**
Average Keypoints	∼4,100 keypoints/image	∼4,900 keypoints/image
Number of Inliers (Pair 3-4)	1,619 points	845 points
Inlier Ratio	∼1.00 (100%)	∼0.98 (98%)
Pipeline Runtime	5.34 seconds	1.72 seconds
Processing Speed	Slow	Fast (∼3× faster)

Table 3: Quantitative comparison between SIFT and ORB

The choice of feature extraction algorithm directly affects the visual quality of the final panorama.

SIFT: Quality-Oriented Approach

- **Seam Smoothness:** Due to the very high number of accurate inliers, the computed homography matrix is highly reliable. This results in nearly invisible seams between images, minimizing ghosting and misalignment artifacts.
- **Robustness:** SIFT maintains stable features even under significant scale and rotation changes, producing a more natural and geometrically consistent panorama.
- **Drawback:** Computationally expensive and not suitable for mobile devices or real-time video processing.

ORB: Performance-Oriented Approach

- **Seam Quality:** Although fast, the number of inliers is noticeably lower (especially in image pair 3-4). This may introduce small accumulated alignment errors, causing slight blur or misalignment in low-texture overlapping regions.
- **Efficiency:** ORB is computationally efficient, making it ideal for real-time applications such as augmented reality (AR) or mobile panorama stitching.
- **Drawback:** Less robust to large scale changes compared to SIFT.

Algorithm	Strengths	Weaknesses
SIFT	High accuracy; Smooth stitching; Robust to scale and rotation changes	Slow processing; High memory usage; Previously subject to patent restrictions
ORB	Very fast; Open-source; Suitable for low-resource devices	Fewer correct matches; Less robust to large scale variations

Table 4: Strengths and weaknesses of SIFT and ORB

5 Conclusion

This report successfully implemented and experimentally evaluated three core problems in computer vision: image denoising, 3D structure reconstruction, and panorama stitching. Based on the obtained results, several important conclusions can be drawn as follows:

5.1 Spatial Domain Image Denoising

The experiments confirm that there is no universal filter suitable for all types of images and noise conditions. The selection of a filtering method should depend on the noise characteristics and the requirement to preserve image details.

- The Gaussian filter demonstrates superior performance in removing Gaussian noise while preserving the natural image structure, achieving the highest PSNR value (approximately 29.72 dB).

- The Mean filter provides a good balance between computational complexity and smoothing performance, although it tends to blur image edges.
- The Laplacian filter is mainly effective for edge enhancement rather than noise removal due to its sensitivity to intensity variations.

5.2 3D Depth Reconstruction (Stereo Vision)

The use of the SGBM (Semi-Global Block Matching) algorithm shows significant improvement in the smoothness and coverage of disparity maps compared to the traditional Block Matching (BM) method.

- The multi-directional optimization mechanism of SGBM effectively handles textureless regions, improving the accuracy of disparity estimation.
- This provides a strong foundation for generating accurate point clouds in 3D space.
- The inverse relationship between disparity and depth is expressed as:

  $$Z \propto \frac{1}{d}$$

  This characteristic is crucial when designing sensor systems based on triangulation principles.

5.3 Panorama Stitching

The comparison between SIFT and ORB highlights the trade-off between accuracy and computational speed.

- SIFT is suitable for applications requiring high accuracy due to its invariance to scale and rotation, reflected in a high inlier ratio.
- ORB is a better choice for real-time systems or mobile devices thanks to its significantly faster processing speed.

6 References

[Your references here]

List all sources (e.g., OpenCV documentation, textbooks) in a consistent format. Use `url` *for online resources and ensure all citations are relevant and properly formatted.*

A Appendix: Python Code

A.1 Part A: Image Filtering

```
from google.colab import drive
drive.mount('/content/drive')
import cv2
```

```python
import numpy as np
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr
import time
import os
from glob import glob
save_dir = '/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Data/Part_a'
os.makedirs(save_dir, exist_ok=True)
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os
from glob import glob
input_dir = "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Data/Part_a"
output_dir = "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results/Part_a"
os.makedirs(output_dir, exist_ok=True)
def pad_image(img, pad):
    return np.pad(img, ((pad,pad),(pad,pad)), mode='reflect')
def mean_filter(img, k=3):
    pad = k//2
    padded = pad_image(img, pad)
    h, w = img.shape
    output = np.zeros_like(img, dtype=np.float32)

    for i in range(h):
        for j in range(w):
            region = padded[i:i+k, j:j+k]
            output[i,j] = np.mean(region)

    return np.clip(output,0,255).astype(np.uint8)

def gaussian_kernel(k=5, sigma=1):
    ax = np.arange(-k//2 + 1., k//2 + 1.)
    xx, yy = np.meshgrid(ax, ax)
    kernel = np.exp(-(xx**2 + yy**2)/(2*sigma**2))
    return kernel / np.sum(kernel)

def gaussian_filter(img, k=5, sigma=1):
    kernel = gaussian_kernel(k, sigma)
    pad = k//2
    padded = pad_image(img, pad)
    h, w = img.shape
    output = np.zeros_like(img, dtype=np.float32)

    for i in range(h):
        for j in range(w):
            region = padded[i:i+k, j:j+k]
            output[i,j] = np.sum(region * kernel)
```

```python

    return np.clip(output,0,255).astype(np.uint8)

def median_filter(img, k=3):
    pad = k//2
    padded = pad_image(img, pad)
    h, w = img.shape
    output = np.zeros_like(img)

    for i in range(h):
        for j in range(w):
            region = padded[i:i+k, j:j+k]
            output[i,j] = np.median(region)

    return output

def laplacian_filter(img):
    kernel = np.array([[0,-1,0],
                       [-1,4,-1],
                       [0,-1,0]])

    pad = 1
    padded = pad_image(img, pad)
    h, w = img.shape
    output = np.zeros_like(img, dtype=np.float32)

    for i in range(h):
        for j in range(w):
            region = padded[i:i+3, j:j+3]
            output[i,j] = np.sum(region * kernel)

    sharpen = img + output
    return np.clip(sharpen,0,255).astype(np.uint8)


image_paths = glob(os.path.join(input_dir, "*"))

for path in image_paths:

    filename = os.path.basename(path)
    name, ext = os.path.splitext(filename)

    print("Processing:", filename)

    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)


    noise = np.random.normal(0, 20, img.shape)
    noisy = np.clip(img + noise,0,255).astype(np.uint8)


```

```
    mean_img = mean_filter(noisy)
    gauss_img = gaussian_filter(noisy)
    median_img = median_filter(noisy)
    laplace_img = laplacian_filter(noisy)


    save_dir = os.path.join(output_dir, name)
    os.makedirs(save_dir, exist_ok=True)


    cv2.imwrite(os.path.join(save_dir,"original.png"), img)
    cv2.imwrite(os.path.join(save_dir,"noisy.png"), noisy)
    cv2.imwrite(os.path.join(save_dir,"mean.png"), mean_img)
    cv2.imwrite(os.path.join(save_dir,"gaussian.png"), gauss_img)
    cv2.imwrite(os.path.join(save_dir,"median.png"), median_img)
    cv2.imwrite(os.path.join(save_dir,"laplacian.png"),
       laplace_img)

    combined =
       np.hstack((img,noisy,mean_img,gauss_img,median_img,laplace_img))
    cv2.imwrite(os.path.join(save_dir,"comparison.png"),
       combined)


    titles = [
        "Original Image",
        "Noisy Image",
        "Mean Filter",
        "Gaussian Filter",
        "Median Filter",
        "Laplacian Sharpen"
    ]

    images = [img, noisy, mean_img, gauss_img, median_img,
       laplace_img]

    plt.figure(figsize=(12,8))
    for i in range(6):
        plt.subplot(2,3,i+1)
        plt.imshow(images[i], cmap='gray')
        plt.title(f"{filename} - {titles[i]}")
        plt.axis('off')

    plt.tight_layout()
    plt.show()

print("Done!")
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim

def compute_metrics(original, filtered):
```

```
    p = psnr(original, filtered)
    s = ssim(original, filtered)
    return p, s
mean_img = mean_filter(noisy)
gauss_img = gaussian_filter(noisy)
median_img = median_filter(noisy)
laplace_img = laplacian_filter(noisy)
# === COMPUTE METRICS ===
filters = {
    "Mean": mean_img,
    "Gaussian": gauss_img,
    "Median": median_img,
    "Laplacian": laplace_img
}

print("\n=== METRICS ===")
for name, f_img in filters.items():
    p, s = compute_metrics(img, f_img)
    print(f"{name}: PSNR={p:.2f}, SSIM={s:.4f}")
with open(os.path.join(save_dir, "metrics.txt"), "w") as f:
    for name, f_img in filters.items():
        p, s = compute_metrics(img, f_img)
        f.write(f"{name}: PSNR={p:.2f}, SSIM={s:.4f}\n")
names = list(filters.keys())
psnr_vals = [compute_metrics(img, filters[n])[0] for n in names]

plt.figure()
plt.bar(names, psnr_vals)
plt.title("PSNR Comparison")
plt.ylabel("PSNR")
plt.savefig(os.path.join(save_dir,
   "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results/Comparis
plt.show()
```

A.2 Part B: 3D Reconstruction

```
from google.colab import drive
drive.mount('/content/drive')
import cv2
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import time, os, glob
os.makedirs("/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Data/Pa
   exist_ok=True)
def show_images(images, titles, rows, cols, save_path=None,
   suptitle=None):
    fig, axes = plt.subplots(rows, cols, figsize=(5*cols,
       5*rows))
    axes = np.atleast_1d(axes).ravel()

```

```
    for ax, img, title in zip(axes, images, titles):
        ax.imshow(img if img.ndim == 2 else cv2.cvtColor(img, cv2.COLOR_BGR2RGB),
                  cmap='gray' if img.ndim == 2 else None)
        ax.set_title(title)
        ax.axis('off')

    if suptitle: fig.suptitle(suptitle, fontsize=14, fontweight='bold')
    plt.tight_layout()

    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
        print(f"Saved: {save_path}")

    plt.show()


print("Imports OK")
LEFT, RIGHT = "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Data/Part_b/left "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Data/Part_b/righ

img_left, img_right = cv2.imread(LEFT), cv2.imread(RIGHT)
assert img_left is not None and img_right is not None, f"Cannot load images: {LEFT}, {RIGHT}"

gray_left  = cv2.cvtColor(img_left,  cv2.COLOR_BGR2GRAY)
gray_right = cv2.cvtColor(img_right, cv2.COLOR_BGR2GRAY)
h, w = gray_left.shape
AUTO_NUM_DISP = max(16, min(int(np.ceil(w / 128)) * 16, 256))

print(f"Image size: {w} x {h}")
print(f"Auto numDisparities: {AUTO_NUM_DISP}")

show_images(
    [img_left, img_right],
    ["Left Image", "Right Image"],
    1, 2,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
    suptitle=f"Stereo Pair ({w}x{h})"
)
def compute_disparity_bm(gray_left, gray_right, num_disp, block_size=15):
    stereo = cv2.StereoBM_create(num_disp, block_size)
    t0 = time.time()
    disp = stereo.compute(gray_left, gray_right)
    dt = time.time() - t0

    vis = cv2.normalize(disp, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
```

```
    return disp, vis, cv2.applyColorMap(vis, cv2.COLORMAP_JET), dt


disp_bm_raw, disp_bm_vis, disp_bm_color, bm_time = compute_disparity_bm(
    gray_left, gray_right, AUTO_NUM_DISP
)

print(f"StereoBM: numDisp={AUTO_NUM_DISP}, time={bm_time:.3f}s")

show_images(
    [img_left, disp_bm_vis, disp_bm_color],
    ["Left Image", "BM Disparity (gray)", "BM Disparity (color)"],
    1, 3,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
    suptitle="StereoBM Disparity Map"
)
def compute_disparity_sgbm(gray_left, gray_right, num_disp, block_size=5):
    stereo = cv2.StereoSGBM_create(
        minDisparity=0,
        numDisparities=num_disp,
        blockSize=block_size,
        P1=8 * 3 * block_size**2,
        P2=32 * 3 * block_size**2,
        disp12MaxDiff=1,
        uniquenessRatio=10,
        speckleWindowSize=100,
        speckleRange=32,
        mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY,
    )
    t0 = time.time()
    disp = stereo.compute(gray_left, gray_right)
    dt = time.time() - t0
    vis = cv2.normalize(disp, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
    color = cv2.applyColorMap(vis, cv2.COLORMAP_JET)
    return disp, vis, color, dt


disp_sgbm_raw, disp_sgbm_vis, disp_sgbm_color, sgbm_time = compute_disparity_sgbm(
    gray_left, gray_right, AUTO_NUM_DISP
)
print(f"StereoSGBM: numDisp={AUTO_NUM_DISP}, time={sgbm_time:.3f}s")

show_images(
    [img_left, disp_sgbm_vis, disp_sgbm_color],
```

```
    ["Left Image", "SGBM Disparity (gray)", "SGBM Disparity (color)"],
    1,
    3,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
    suptitle="StereoSGBM Disparity Map",
)
def rectify_uncalibrated_stereo(img_left, img_right):
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(img_left, None)
    kp2, des2 = sift.detectAndCompute(img_right, None)

    flann = cv2.FlannBasedMatcher(dict(algorithm=1, trees=5), dict(checks=50))
    matches = flann.knnMatch(des1, des2, k=2)

    pts1, pts2 = [], []
    for m, n in matches:
        if m.distance < 0.7 * n.distance:
            pts1.append(kp1[m.queryIdx].pt)
            pts2.append(kp2[m.trainIdx].pt)

    pts1, pts2 = np.int32(pts1), np.int32(pts2)
    print(f"Matches: {len(pts1)}")

    F, mask = cv2.findFundamentalMat(pts1, pts2, cv2.FM_RANSAC, 3, 0.99)
    if mask is not None:
        pts1, pts2 = pts1[mask.ravel()==1], pts2[mask.ravel()==1]
    print(f"Inliers: {len(pts1)}")

    h, w = img_left.shape[:2]
    _, H1, H2 = cv2.stereoRectifyUncalibrated(pts1.astype(np.float32), pts2.astype(np.float32), F, (w, h))

    return (
        cv2.warpPerspective(img_left, H1, (w, h)),
        cv2.warpPerspective(img_right, H2, (w, h)),
        H1, H2
    )


#         Rectify
print("Rectifying...")
rect_left, rect_right, H1, H2 = rectify_uncalibrated_stereo(gray_left, gray_right)

show_images(
    [rect_left, rect_right],
    ["Left (Rectified)", "Right (Rectified)"],
```

```
    1, 2,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
)

#         Disparity (SGBM)
print("Computing disparity...")
block_size, num_disp = 5, 256

stereo = cv2.StereoSGBM_create(
    0, num_disp, block_size,
    P1=8*3*block_size**2,
    P2=32*3*block_size**2,
    disp12MaxDiff=1,
    uniquenessRatio=10,
    speckleWindowSize=100,
    speckleRange=32,
    mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY
)

disp = stereo.compute(rect_left, rect_right).astype(np.float32)
   / 16.0
disp_vis = cv2.normalize(disp, None, 0, 255, cv2.NORM_MINMAX,
   cv2.CV_8U)

show_images(
    [disp_vis],
    ["Disparity (Rectified)"],
    1, 1,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
)
def rectify_uncalibrated_stereo(img_left, img_right):
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(img_left, None)
    kp2, des2 = sift.detectAndCompute(img_right, None)

    flann = cv2.FlannBasedMatcher(dict(algorithm=1, trees=5),
       dict(checks=50))
    matches = flann.knnMatch(des1, des2, k=2)

    pts1, pts2 = [], []
    for m, n in matches:
        if m.distance < 0.7 * n.distance:
            pts1.append(kp1[m.queryIdx].pt)
            pts2.append(kp2[m.trainIdx].pt)

    pts1, pts2 = np.int32(pts1), np.int32(pts2)
    print(f"Matches: {len(pts1)}")

    F, mask = cv2.findFundamentalMat(pts1, pts2, cv2.FM_RANSAC,
       3, 0.99)
    if mask is not None:
```

```
        pts1, pts2 = pts1[mask.ravel()==1], pts2[mask.ravel()==1]
    print(f"Inliers: {len(pts1)}")

    h, w = img_left.shape[:2]
    _, H1, H2 = cv2.stereoRectifyUncalibrated(pts1.astype(np.float32), pts2.astype(np.float32), F, (w, h))

    return (
        cv2.warpPerspective(img_left, H1, (w, h)),
        cv2.warpPerspective(img_right, H2, (w, h)),
        H1, H2
    )


#         Rectify
print("Rectifying...")
rect_left, rect_right, H1, H2 = rectify_uncalibrated_stereo(gray_left, gray_right)

show_images(
    [rect_left, rect_right],
    ["Left (Rectified)", "Right (Rectified)"],
    1, 2,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
)

#         Disparity (SGBM)
print("Computing disparity...")
block_size, num_disp = 5, 256

stereo = cv2.StereoSGBM_create(
    0, num_disp, block_size,
    P1=8*3*block_size**2,
    P2=32*3*block_size**2,
    disp12MaxDiff=1,
    uniquenessRatio=10,
    speckleWindowSize=100,
    speckleRange=32,
    mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY
)

disp = stereo.compute(rect_left, rect_right).astype(np.float32) / 16.0
disp_vis = cv2.normalize(disp, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)

show_images(
    [disp_vis],
    ["Disparity (Rectified)"],
    1, 1,
```

```
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
)
def rectify_uncalibrated_stereo(img_left, img_right):
    sift = cv2.SIFT_create()
    kp1, des1 = sift.detectAndCompute(img_left, None)
    kp2, des2 = sift.detectAndCompute(img_right, None)

    flann = cv2.FlannBasedMatcher(dict(algorithm=1, trees=5),
       dict(checks=50))
    matches = flann.knnMatch(des1, des2, k=2)

    pts1, pts2 = [], []
    for m, n in matches:
        if m.distance < 0.7 * n.distance:
            pts1.append(kp1[m.queryIdx].pt)
            pts2.append(kp2[m.trainIdx].pt)

    pts1, pts2 = np.int32(pts1), np.int32(pts2)
    print(f"Matches: {len(pts1)}")

    F, mask = cv2.findFundamentalMat(pts1, pts2, cv2.FM_RANSAC,
       3, 0.99)
    if mask is not None:
        pts1, pts2 = pts1[mask.ravel()==1], pts2[mask.ravel()==1]
    print(f"Inliers: {len(pts1)}")

    h, w = img_left.shape[:2]
    _, H1, H2 =
       cv2.stereoRectifyUncalibrated(pts1.astype(np.float32),
       pts2.astype(np.float32), F, (w, h))

    return (
        cv2.warpPerspective(img_left, H1, (w, h)),
        cv2.warpPerspective(img_right, H2, (w, h)),
        H1, H2
    )


#         Rectify
print("Rectifying...")
rect_left, rect_right, H1, H2 =
   rectify_uncalibrated_stereo(gray_left, gray_right)

show_images(
    [rect_left, rect_right],
    ["Left (Rectified)", "Right (Rectified)"],
    1, 2,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
)

#         Disparity (SGBM)
```

```python
print("Computing disparity...")
block_size, num_disp = 5, 256

stereo = cv2.StereoSGBM_create(
    0, num_disp, block_size,
    P1=8*3*block_size**2,
    P2=32*3*block_size**2,
    disp12MaxDiff=1,
    uniquenessRatio=10,
    speckleWindowSize=100,
    speckleRange=32,
    mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY
)

disp = stereo.compute(rect_left, rect_right).astype(np.float32) / 16.0
disp_vis = cv2.normalize(disp, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)

show_images(
    [disp_vis],
    ["Disparity (Rectified)"],
    1, 1,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
)
disp = disp_sgbm_raw.astype(np.float32) / 16.0

valid_disp = disp[disp > 0]
MIN_DISP = max(1.0, np.percentile(valid_disp, 5))

disp_range = np.percentile(valid_disp, 95) - MIN_DISP
DEPTH_SCALE = (w * 0.3) / max(disp_range, 1)

print(f"Range: [{valid_disp.min():.1f}, {valid_disp.max():.1f}]")
print(f"MIN_DISP: {MIN_DISP:.1f}, DEPTH_SCALE: {DEPTH_SCALE:.1f}")
mask = disp >= MIN_DISP
u, v = np.meshgrid(np.arange(w), np.arange(h))

points = np.stack([
    u[mask].astype(np.float32),
    -v[mask].astype(np.float32),
    disp[mask] * DEPTH_SCALE
], axis=1)

colors = cv2.cvtColor(img_left, cv2.COLOR_BGR2RGB)[mask] / 255.0

print(f"\nPoints: {len(points)}")
print(f"X:[{points[:,0].min():.0f},{points[:,0].max():.0f}] "
      f"Y:[{points[:,1].min():.0f},{points[:,1].max():.0f}] "
      f"Z:[{points[:,2].min():.0f},{points[:,2].max():.0f}]")
```

```python
ply_path = "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results/Part_b/p
# Use 'mask' instead of 'valid' and get unscaled RGB values (0-255)
rgb_points = cv2.cvtColor(img_left, cv2.COLOR_BGR2RGB)[mask]
# Extract x, y, z coordinates from the 'points' array
x_vals = points[:, 0]
y_vals = points[:, 1]
z_vals = points[:, 2]
data = np.column_stack([x_vals, y_vals, z_vals, rgb_points[:, 0], rgb_points[:, 1], rgb_points[:, 2]])

header = (
    f"ply\nformat ascii 1.0\nelement vertex {len(data)}\n"
    f"property float x\nproperty float y\nproperty float z\n"
    f"property uchar red\nproperty uchar green\nproperty uchar blue\n"
    f"end_header"
)
np.savetxt(ply_path, data, fmt="%.1f %.1f %.1f %d %d %d", header=header, comments="")
print(f"PLY saved: {ply_path} ({len(data)} vertices)")
step = max(1, len(points) // 10000)
pts = points[::step]
cols = colors[::step]
print(f"Plotting {len(pts)} points (subsampled {step}x)")

fig = plt.figure(figsize=(16, 6))
for i, (title, elev, azim) in enumerate(
    [("Front View", 0, -90), ("Top View", 90, -90), ("Perspective View", 30, -60)]
):
    ax = fig.add_subplot(1, 3, i + 1, projection="3d")
    ax.scatter(pts[:, 0], pts[:, 1], pts[:, 2], c=cols, s=0.3)
    ax.set_title(title)
    ax.view_init(elev=elev, azim=azim)

plt.suptitle("3D Point Cloud from Stereo Disparity", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results
dpi=150, bbox_inches="tight")
print("Saved: /content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results/Part_b/p
plt.show()
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection="3d")
z_norm = (pts[:, 2] - pts[:, 2].min()) / (pts[:, 2].max() - pts[:, 2].min() + 1e-8)
sc = ax.scatter(pts[:, 0], pts[:, 1], pts[:, 2], c=z_norm, cmap="plasma", s=0.5)
```

```
ax.set_title("Depth-Colored 3D Point Cloud", fontsize=14, fontweight="bold")
ax.view_init(elev=25, azim=-60)
fig.colorbar(sc, shrink=0.6, label="Relative Depth")
plt.tight_layout()
plt.savefig(
    "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results/Part_b/
        dpi=150, bbox_inches="tight"
)
print("Saved: /content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results/Part_b/p
plt.show()
show_images(
    [img_left, disp_bm_color, disp_sgbm_color],
    ["Left Image", f"StereoBM ({bm_time:.3f}s)", f"StereoSGBM ({sgbm_time:.3f}s)"],
    1,
    3,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
    suptitle="BM vs SGBM Disparity Maps",
)
block_sizes = [5, 9, 15]
bm_res, sgbm_res, bm_t, sgbm_t = [], [], [], []

for bs in block_sizes:
    bm_res.append(compute_disparity_bm(gray_left, gray_right, AUTO_NUM_DISP, bs)[2])
    sgbm_res.append(compute_disparity_sgbm(gray_left, gray_right, AUTO_NUM_DISP, bs)[2])
    bm_t.append(compute_disparity_bm(gray_left, gray_right, AUTO_NUM_DISP, bs)[3])
    sgbm_t.append(compute_disparity_sgbm(gray_left, gray_right, AUTO_NUM_DISP, bs)[3])

fig, ax = plt.subplots(2, 3, figsize=(15, 8))

for j, bs in enumerate(block_sizes):
    for i, (res, t, name) in enumerate([(bm_res, bm_t, "BM"), (sgbm_res, sgbm_t, "SGBM")]):
        ax[i, j].imshow(cv2.cvtColor(res[j], cv2.COLOR_BGR2RGB))
        ax[i, j].set_title(f"{name} (block={bs}, {t[j]:.3f}s)")
        ax[i, j].axis("off")

plt.suptitle("Block Size Comparison", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results
    dpi=150, bbox_inches="tight")
plt.show()
```

A.3 Part C: mage Stitching

```
from google.colab import drive
drive.mount('/content/drive')
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time
import os
os.makedirs("/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Data/Pa
    exist_ok=True)
import cv2
import os

#     ng      d n  data
save_dir =
    "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Data/Part_c"
os.makedirs(save_dir, exist_ok=True)

# Load panorama
img =
    cv2.imread("/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Data/
height, width, _ = img.shape

num_images = 4
crop_width = int(width * 0.40)
stride = (width - crop_width) // (num_images - 1)

for i in range(num_images):
    start_x = i * stride
    end_x = start_x + crop_width

    crop = img[:, start_x:end_x]

    filename = os.path.join(save_dir, f"img{i+1}.jpg")  # <--
        l u v o Data
    cv2.imwrite(filename, crop)

    print(f"Saved {filename}  (Columns: {start_x}     {end_x})")
plt.rcParams["figure.figsize"] = (16, 10)
plt.rcParams["figure.dpi"] = 100
plt.rcParams["font.size"] = 10
def show_images(images, titles, rows, cols, save_path=None,
    suptitle=None):
    fig, axes = plt.subplots(rows, cols, figsize=(5*cols,
        5*rows))
    if suptitle:
        fig.suptitle(suptitle)

    axes = np.array(axes).ravel()

    for ax, img, title in zip(axes, images, titles):
```

```
        ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB) if img.ndim==3 else img,
                  cmap=None if img.ndim==3 else "gray")
        ax.set_title(title)
        ax.axis("off")

    for ax in axes[len(images):]:
        ax.axis("off")

    plt.tight_layout()

    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches="tight")

    plt.show()
import os
image_folder_path = "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Data/Part_c"

images = []
for i in range(1, 5):
    img_path = os.path.join(image_folder_path, f"img{i}.jpg")
    img = cv2.imread(img_path)
    if img is None:
        raise FileNotFoundError(f"img{i}.jpg not found in {image_folder_path}/")
    images.append(img)
    print(f"img{i}.jpg: {img.shape}")
results_dir = "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results/Part_c"
os.makedirs(results_dir, exist_ok=True)

show_images(
    images,
    [f"Image {i+1}" for i in range(4)],
    1,
    4,
    save_path=os.path.join(results_dir, "input_images.png"),
    suptitle="Input Images for Stitching",
)
def detect_features(images):
    def run(det):
        t0 = time.time()
        kp, des = zip(*[
            det.detectAndCompute(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), None)
            for img in images
        ])
        return list(kp), list(des), time.time() - t0

    kp_sift, des_sift, time_sift = run(cv2.SIFT_create())
```

```
    kp_orb,  des_orb,  time_orb  = run(cv2.ORB_create(5000))

    return (kp_sift, des_sift, time_sift), (kp_orb, des_orb,
        time_orb)
    (sift, orb) = detect_features(images)

kp_sift, des_sift, time_sift = sift
kp_orb,  des_orb,  time_orb  = orb
print("SIFT Detection:")
for i, kp in enumerate(kp_sift):
    print(f"  Image {i+1}: {len(kp)} keypoints")
print(f"  Time: {time_sift:.3f}s")

print("\nORB Detection:")
for i, kp in enumerate(kp_orb):
    print(f"  Image {i+1}: {len(kp)} keypoints")
print(f"  Time: {time_orb:.3f}s")
kp_imgs_sift = []
kp_imgs_orb = []
for i in range(4):
    kp_img_s = cv2.drawKeypoints(
        images[i],
        kp_sift[i],
        None,
        color=(0, 255, 0),
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS,
    )
    kp_imgs_sift.append(kp_img_s)

    kp_img_o = cv2.drawKeypoints(
        images[i],
        kp_orb[i],
        None,
        color=(0, 0, 255),
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS,
    )
    kp_imgs_orb.append(kp_img_o)

show_images(
    kp_imgs_sift,
    [f"SIFT - Img{i+1} ({len(kp_sift[i])} kp)" for i in
        range(4)],
    1,
    4,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
    suptitle="SIFT Keypoints",
)

show_images(
    kp_imgs_orb,
    [f"ORB - Img{i+1} ({len(kp_orb[i])} kp)" for i in range(4)],
```

```
    1,
    4,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
    suptitle="ORB Keypoints",
)
def match_features(des1, des2, detector_type="SIFT", ratio=0.75):
    """Match features between two images using BFMatcher +
       Lowe's ratio test."""
    if des1 is None or des2 is None:
        return [], 0, 0.0

    if detector_type == "SIFT":
        bf = cv2.BFMatcher(cv2.NORM_L2)
    else:
        bf = cv2.BFMatcher(cv2.NORM_HAMMING)

    t0 = time.time()
    matches = bf.knnMatch(des1, des2, k=2)
    match_time = time.time() - t0

    # Lowe's ratio test
    good = []
    for m, n in matches:
        if m.distance < ratio * n.distance:
            good.append(m)

    return good, len(matches), match_time


pairs = [(0, 1), (1, 2), (2, 3)]

print("SIFT Matching:")
matches_sift = {}
total_match_time_sift = 0
for i, j in pairs:
    good, total, mt = match_features(des_sift[i], des_sift[j],
       "SIFT")
    matches_sift[(i, j)] = good
    total_match_time_sift += mt
    print(f"  Pair {i+1}-{j+1}: {total} raw     {len(good)} good
       matches")
print(f"  Total time: {total_match_time_sift:.3f}s")

print("\nORB Matching:")
matches_orb = {}
total_match_time_orb = 0
for i, j in pairs:
    good, total, mt = match_features(des_orb[i], des_orb[j],
       "ORB")
    matches_orb[(i, j)] = good
    total_match_time_orb += mt
```

```
    print(f"  Pair {i+1}-{j+1}: {total} raw     {len(good)} good matches")
print(f"  Total time: {total_match_time_orb:.3f}s")
for i, j in pairs:
    match_img = cv2.drawMatches(
        images[i],
        kp_sift[i],
        images[j],
        kp_sift[j],
        matches_sift[(i, j)][:50],
        None,
        matchColor=(0, 255, 0),
        singlePointColor=(255, 0, 0),
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS,
    )
    show_images(
        [match_img],
        [f"SIFT Matches: Img{i+1}     Img{j+1} ({len(matches_sift[(i,j)])} good)"],
        1,
        1,
        save_path=f"/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/
    )
for i, j in pairs:
    match_img = cv2.drawMatches(
        images[i],
        kp_orb[i],
        images[j],
        kp_orb[j],
        matches_orb[(i, j)][:50],
        None,
        matchColor=(0, 0, 255),
        singlePointColor=(255, 0, 0),
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS,
    )
    show_images(
        [match_img],
        [f"ORB Matches: Img{i+1}     Img{j+1} ({len(matches_orb[(i,j)])} good)"],
        1,
        1,
        save_path=f"/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/
    )
def compute_homography(kp1, kp2, matches, thresh=5.0):
    if len(matches) < 4:
        print(f"WARNING: Only {len(matches)} matches")
        return None, 0, None

    src = np.float32([kp1[m.queryIdx].pt for m in matches]).reshape(-1,1,2)
```

```
    dst = np.float32([kp2[m.trainIdx].pt for m in matches]).reshape(-1,1,2)

    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, thresh)
    return H, int(mask.sum()) if mask is not None else 0, mask
print("SIFT Homographies:")
H_sift = {}
inliers_sift = {}
for i, j in pairs:
    H, inl, mask = compute_homography(kp_sift[i], kp_sift[j], matches_sift[(i, j)])
    H_sift[(i, j)] = H
    inliers_sift[(i, j)] = inl
    print(f"  Pair {i+1}-{j+1}: {inl} inliers / {len(matches_sift[(i,j)])} matches")
    if H is not None:
        print(f"    H = \n{H}")

print("\nORB Homographies:")
H_orb = {}
inliers_orb = {}
for i, j in pairs:
    H, inl, mask = compute_homography(kp_orb[i], kp_orb[j], matches_orb[(i, j)])
    H_orb[(i, j)] = H
    inliers_orb[(i, j)] = inl
    print(f"  Pair {i+1}-{j+1}: {inl} inliers / {len(matches_orb[(i,j)])} matches")
    if H is not None:
        print(f"    H = \n{H}")
def stitch_pair(img1, img2, t="SIFT", ratio=0.75):
    det = cv2.SIFT_create() if t=="SIFT" else cv2.ORB_create(5000)
    bf  = cv2.BFMatcher(cv2.NORM_L2 if t=="SIFT" else cv2.NORM_HAMMING)

    kp1, des1 = det.detectAndCompute(cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY), None)
    kp2, des2 = det.detectAndCompute(cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY), None)
    if des1 is None or des2 is None: return img2

    good = [m for m,n in bf.knnMatch(des1,des2,2) if m.distance < ratio*n.distance]
    if len(good) < 4: return img2

    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1,1,2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1,1,2)
    H,_ = cv2.findHomography(src,dst,cv2.RANSAC,5.0)
```

```
    if H is None: return img2

    h1,w1 = img1.shape[:2]; h2,w2 = img2.shape[:2]
    c1 = np.float32([[0,0],[w1,0],[w1,h1],[0,h1]]).reshape(-1,1,2)
    c2 = np.float32([[0,0],[w2,0],[w2,h2],[0,h2]]).reshape(-1,1,2)

    warped = cv2.perspectiveTransform(c1,H)
    all_c = np.concatenate([warped,c2])
    x_min,y_min = np.int32(all_c.min(0).ravel())-10
    x_max,y_max = np.int32(all_c.max(0).ravel())+10

    T = np.array([[1,0,-x_min],[0,1,-y_min],[0,0,1]])
    res = cv2.warpPerspective(img1, T@H, (x_max-x_min, y_max-y_min))

    y,x = -y_min, -x_min
    roi = res[y:y+h2, x:x+w2]
    mask = (roi>0).any(2) & (img2>0).any(2)

    res[y:y+h2, x:x+w2] = img2
    res[y:y+h2, x:x+w2][mask] = ((roi[mask].astype(float)+img2[mask])/2).astype(np.uint8)

    return res
def crop_black_borders(img):
    g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    pts = cv2.findNonZero((g>0).astype(np.uint8))
    if pts is None: return img
    x,y,w,h = cv2.boundingRect(pts)
    return img[y:y+h, x:x+w]
print("=== SIFT Stitching ===")
t0 = time.time()

result_sift = images[0].copy()
for i in range(1, 4):
    print(f"  Stitching image {i+1}...")
    result_sift = stitch_pair(result_sift, images[i], "SIFT")
    print(f"  Intermediate result shape: {result_sift.shape}")

result_sift = crop_black_borders(result_sift)
time_sift_total = time.time() - t0

cv2.imwrite("/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results
    result_sift)
print(f"\nSIFT Panorama: {result_sift.shape}, Time: {time_sift_total:.2f}s")

show_images(
    [result_sift],
```

```
    ["SIFT Panorama"],
    1,
    1,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
    suptitle="Final Panorama     SIFT",
)
print("=== ORB Stitching ===")
t0 = time.time()

result_orb = images[0].copy()
for i in range(1, 4):
    print(f"  Stitching image {i+1}...")
    result_orb = stitch_pair(result_orb, images[i], "ORB")
    print(f"  Intermediate result shape: {result_orb.shape}")

result_orb = crop_black_borders(result_orb)
time_orb_total = time.time() - t0

cv2.imwrite("/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results
    result_orb)
print(f"\nORB Panorama: {result_orb.shape}, Time:
    {time_orb_total:.2f}s")

show_images(
    [result_orb],
    ["ORB Panorama"],
    1,
    1,
    save_path="/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Resul
    suptitle="Final Panorama     ORB",
)
print(f"{'='*90}")
print(f"{'SIFT vs ORB     Detailed Comparison':^90}")
print(f"{'='*90}")

print(f"\n{'Pair':<8} {'Metric':<20} {'SIFT':>12} {'ORB':>12}")
print(f"{'-'*55}")

for i, j in pairs:
    print(
        f'{i+1}-{j+1:<5} {"Keypoints (img1)":<20}
            {len(kp_sift[i]):>12} {len(kp_orb[i]):>12}'
    )
    print(
        f'{"" :<8} {"Keypoints (img2)":<20}
            {len(kp_sift[j]):>12} {len(kp_orb[j]):>12}'
    )
    print(
        f'{"" :<8} {"Good Matches":<20}
            {len(matches_sift[(i,j)]):>12}
            {len(matches_orb[(i,j)]):>12}'
```

```
    )
    print(
        f'{"" :<8} {"RANSAC Inliers":<20} {inliers_sift[(i,j)]:>12} {inliers_orb[(i,j)]:>12}'
    )
    print()

print(f"\n{'Timing':<25} {'SIFT':>12} {'ORB':>12}")
print(f"{'-'*50}")
print(f"{'Detection':<25} {time_sift:>11.3f}s {time_orb:>11.3f}s")
print(
    f"{'Matching':<25} {total_match_time_sift:>11.3f}s {total_match_time_orb:>11.3f}s"
)
print(f"{'Total Pipeline':<25} {time_sift_total:>11.3f}s {time_orb_total:>11.3f}s")
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

axes[0].imshow(cv2.cvtColor(result_sift, cv2.COLOR_BGR2RGB))
axes[0].set_title(f"SIFT Panorama ({time_sift_total:.2f}s)", fontsize=14)
axes[0].axis("off")

axes[1].imshow(cv2.cvtColor(result_orb, cv2.COLOR_BGR2RGB))
axes[1].set_title(f"ORB Panorama ({time_orb_total:.2f}s)", fontsize=14)
axes[1].axis("off")

plt.suptitle("SIFT vs ORB     Panorama Comparison", fontsize=16, fontweight="bold")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results
    dpi=150, bbox_inches="tight")
print("Saved: /content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results/Part_c/pa
plt.show()
print("\n=== SUMMARY COMPARISON ===")

for i, j in pairs:
    print(f"\nPair {i+1}-{j+1}:")
    print(f"SIFT     Matches: {len(matches_sift[(i,j)])}, Inliers: {inliers_sift[(i,j)]}")
    print(f"ORB      Matches: {len(matches_orb[(i,j)])}, Inliers: {inliers_orb[(i,j)]}")
    ratio_sift = inliers_sift[(i,j)] / len(matches_sift[(i,j)])
ratio_orb  = inliers_orb[(i,j)] / len(matches_orb[(i,j)])
print(f"SIFT Inlier Ratio: {ratio_sift:.2f}")
print(f"ORB  Inlier Ratio: {ratio_orb:.2f}")
labels = ["SIFT", "ORB"]
times = [time_sift_total, time_orb_total]
```

```python

plt.bar(labels, times)
plt.title("Processing Time Comparison")
plt.ylabel("Seconds")
plt.savefig(os.path.join(results_dir, "time_comparison.png"))
plt.show()
summary_path = os.path.join(results_dir,
    "/content/drive/MyDrive/Midterm_23070471_Nguyen_Tuan_Minh/Results/Compari

with open(summary_path, "w") as f:
    f.write("=== SIFT vs ORB COMPARISON ===\n\n")

    for i, j in pairs:
        f.write(f"Pair {i+1}-{j+1}:\n")

        sift_matches = len(matches_sift[(i,j)])
        orb_matches  = len(matches_orb[(i,j)])

        sift_inliers = inliers_sift[(i,j)]
        orb_inliers  = inliers_orb[(i,j)]

        sift_ratio = sift_inliers / sift_matches if sift_matches
            > 0 else 0
        orb_ratio  = orb_inliers / orb_matches if orb_matches >
            0 else 0

        f.write(f"  SIFT     Matches: {sift_matches}, Inliers:
            {sift_inliers}, Ratio: {sift_ratio:.2f}\n")
        f.write(f"  ORB      Matches: {orb_matches}, Inliers:
            {orb_inliers}, Ratio: {orb_ratio:.2f}\n\n")

    f.write("=== TIMING ===\n")
    f.write(f"SIFT Detection: {time_sift:.3f}s\n")
    f.write(f"ORB Detection: {time_orb:.3f}s\n")
    f.write(f"SIFT Matching: {total_match_time_sift:.3f}s\n")
    f.write(f"ORB Matching: {total_match_time_orb:.3f}s\n")
    f.write(f"SIFT Total: {time_sift_total:.3f}s\n")
    f.write(f"ORB Total: {time_orb_total:.3f}s\n")

print("Saved:", summary_path)
```